Terraform-02

Execute all the templates shown in the video

Variables:

>> Method 1 : initialize the variable values in the file

We have various types of variable but we can discuss on string,number.

We can execute this we want a main.tf file and we want one variable.tf file. I can use existing main.tf file.

> i will create a variable.tf

On that i will add the below content

```
variable "filename" {
default = "pets.txt"
type = string
}
variable "content" {
  default = "i love dogs"
}
```

In my main.tf file i will declare these variables

```
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "petname" {
prefix = "MR"
separator = "."
length = "1"
}
```

After that initialize the file

terraform init

After that plan the file

terraform plan

After that apply

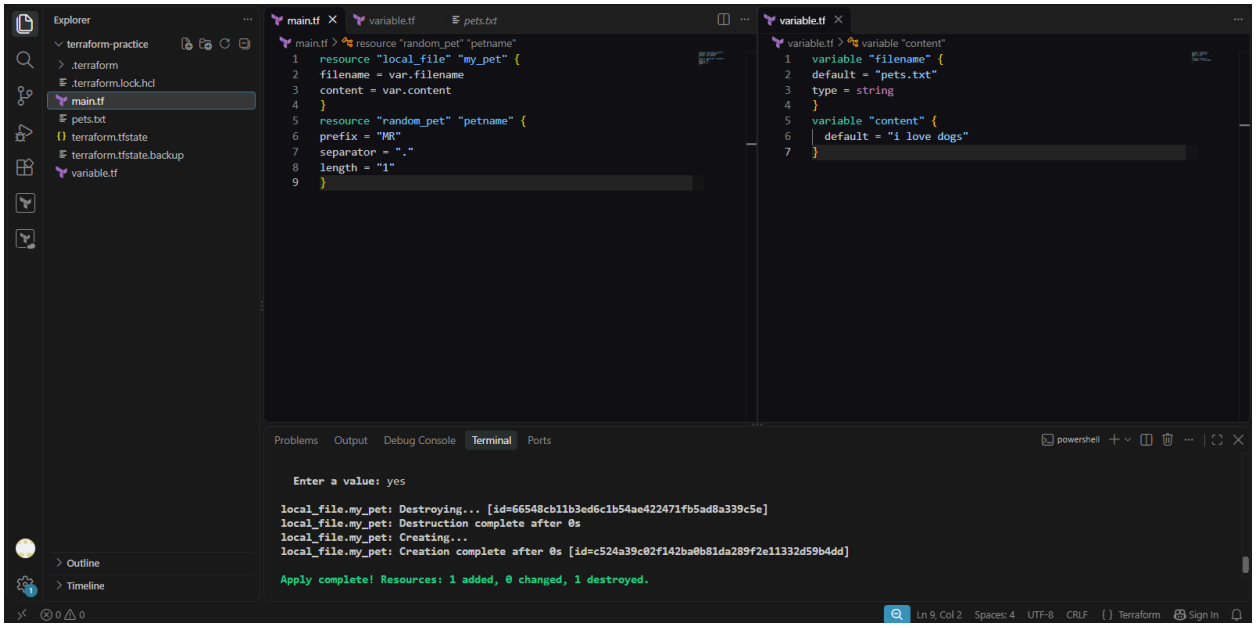
terraform apply

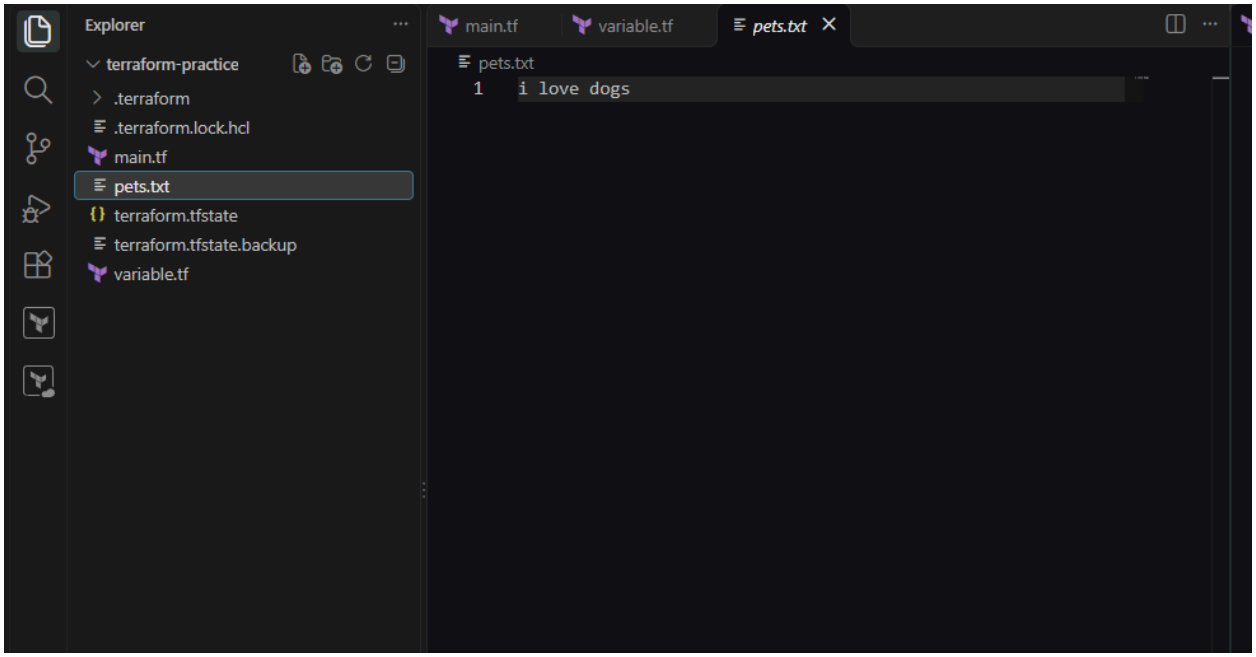
We can get the output

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

It will get the output as

I love dogs



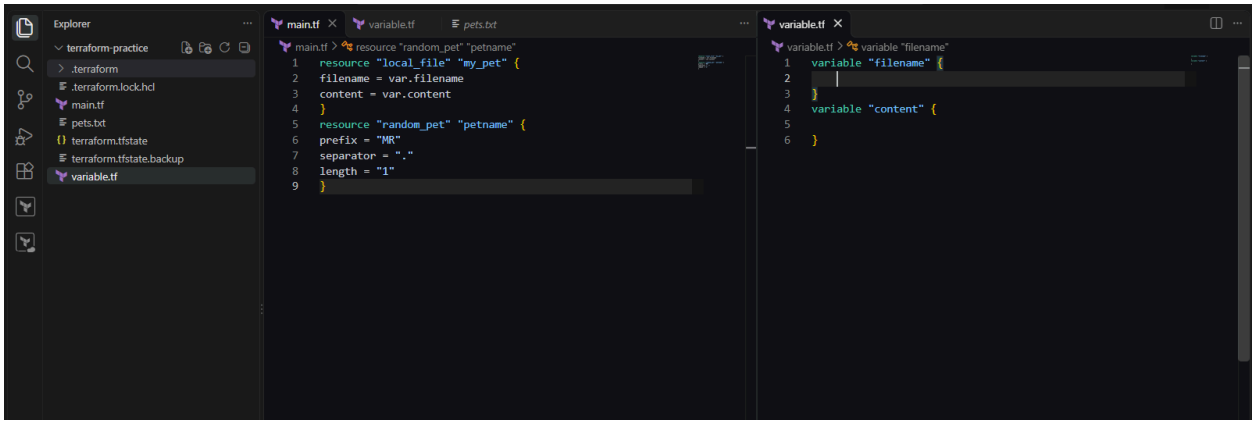


>> Method 2: Command line Arguments

In this we can declare the empty variable.tf file

Type - 1:

We can apply on runtime it will ask the values



If i can execute this it will ask the enter the values

Execute this : terraform apply

```
PS C:\terraform-practice> terraform apply
var.content
  Enter a value: i love karthik hairstyle

var.filename
  Enter a value: pets.txt

random_pet.petname: Refreshing state... [id=MR.owl]
local_file.my_pet: Refreshing state... [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]
```

The the output is : i love karthik hairstyle in the pets.txt file



Type - 2 :

Entering the values on the apply time

The syntax is terraform apply -var “tagname=” -var “taganme=”.

Remember there is no gap between the taganems

```
PS C:\terraform-practice> terraform apply -var "filename=dog.txt" -var "content=ilovedogs"
random_pet.petname: Refreshing state... [id=MR.owl]
local_file.my_pet: Refreshing state... [id=0c088ec7968c2946300476d8c9e70725ca05e6e2]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # local_file.my_pet must be replaced
-/+ resource "local_file" "my_pet" {
      ~ content              = "i love karthik hairstyle" -> "ilovedogs" # forces replacement
      ~ content_base64sha256 = "uW1D5Bb9nxIjqfNBFg+ObJIWeVEDRNNwwWkkKOwKPRc=" -> (known after apply)
      ~ content_base64sha512 = "9nhAYH2pNJEizvFS5MrrU15ch1ZiKmzpdCw/xKsvnPkY1D5c989q2Wqe3gA/VvJiAiCv1u5IR7tEOZ3FSkM7QQ==" -> (known after apply)
      ~ content_md5          = "2a246edf27c2147a043368228358b446" -> (known after apply)
      ~ content_sha1         = "0c088ec7968c2946300476d8c9e70725ca05e6e2" -> (known after apply)
      ~ content_sha256       = "b96d43e416fd9f1223a9f341160f8e6c921679510344d370c1692428ec0a3d17" -> (known after apply)
      ~ content_sha512       = "f67840607da9349122cef152e4caeb535e5c8756622a6ce9742c3fc4ab2f9cf918943e5cf7cf6ad96a9ede003f56f2620220afd6ee4847bb44399dc54
a433b41" -> (known after apply)
      ~ filename             = "pets.txt" -> "dog.txt" # forces replacement
      ~ id                   = "0c088ec7968c2946300476d8c9e70725ca05e6e2" -> (known after apply)
        # (2 unchanged attributes hidden)
    }

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

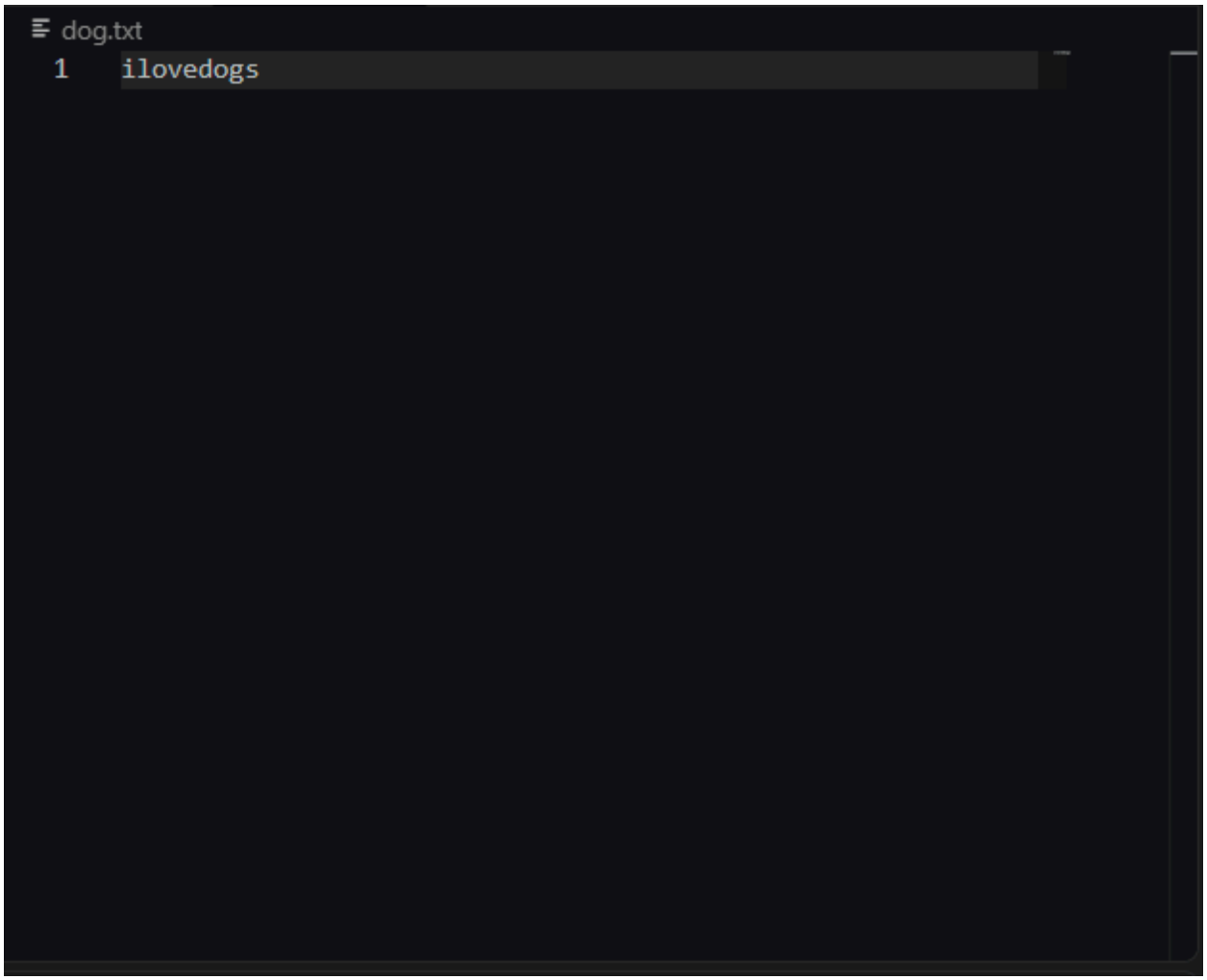
  Enter a value: yes

local_file.my_pet: Destroying... [id=0c088ec7968c2946300476d8c9e70725ca05e6e2]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=0d6dbea8f6b5a834e174fed7b07bcbb81dc3919f]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
```

After that we get output as

Ilovedogs in a dog.txt file



Life Cycle rules

Mainly we have 3 types of life cycles are there

1. Create_before_destroy
2. Prevent_destroy
3. ignore_changes

1.Create_before_changes

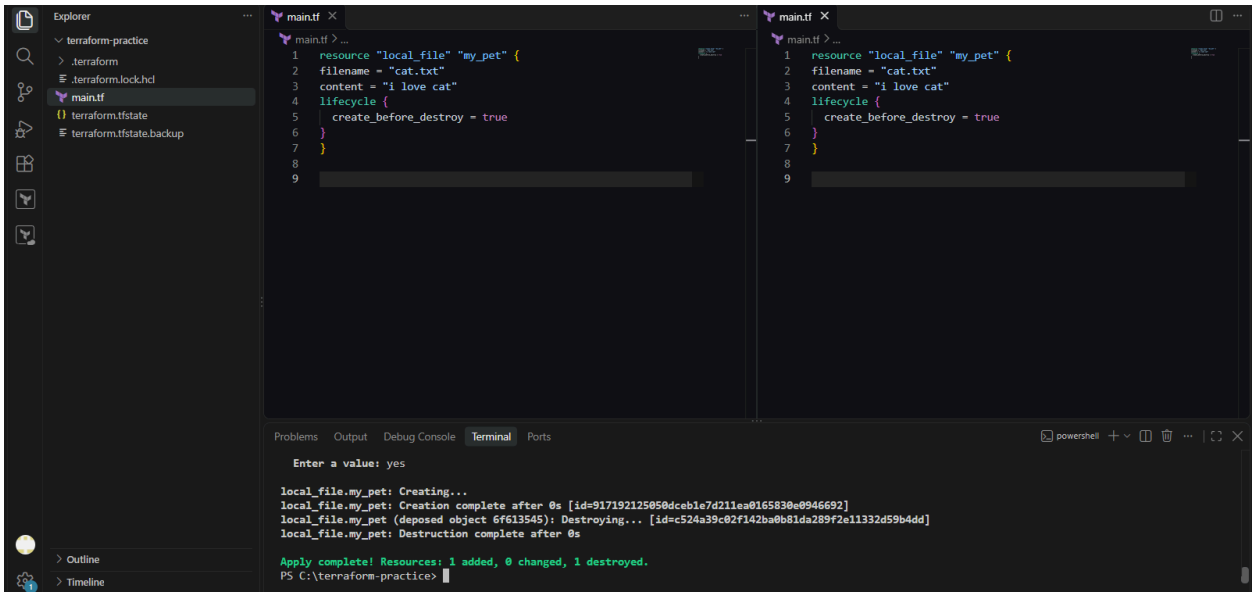
It means we can create a file before destroying

The syntax is

```
Lifecycle {
   create_before_destroy=true
}
```

```tf
resource "local_file" "my_pet" {
filename = "cat.txt"
content = "i love cats"
lifecycle {
  create_before_destroy = true
}
}
```

Then we can apply we can first creation after that it will destroy



2. Prevent_destroy

If we dont want to delete for any reason . That means if we enter or modify any changes it will give me the error as instance can not destroed

The syntax:

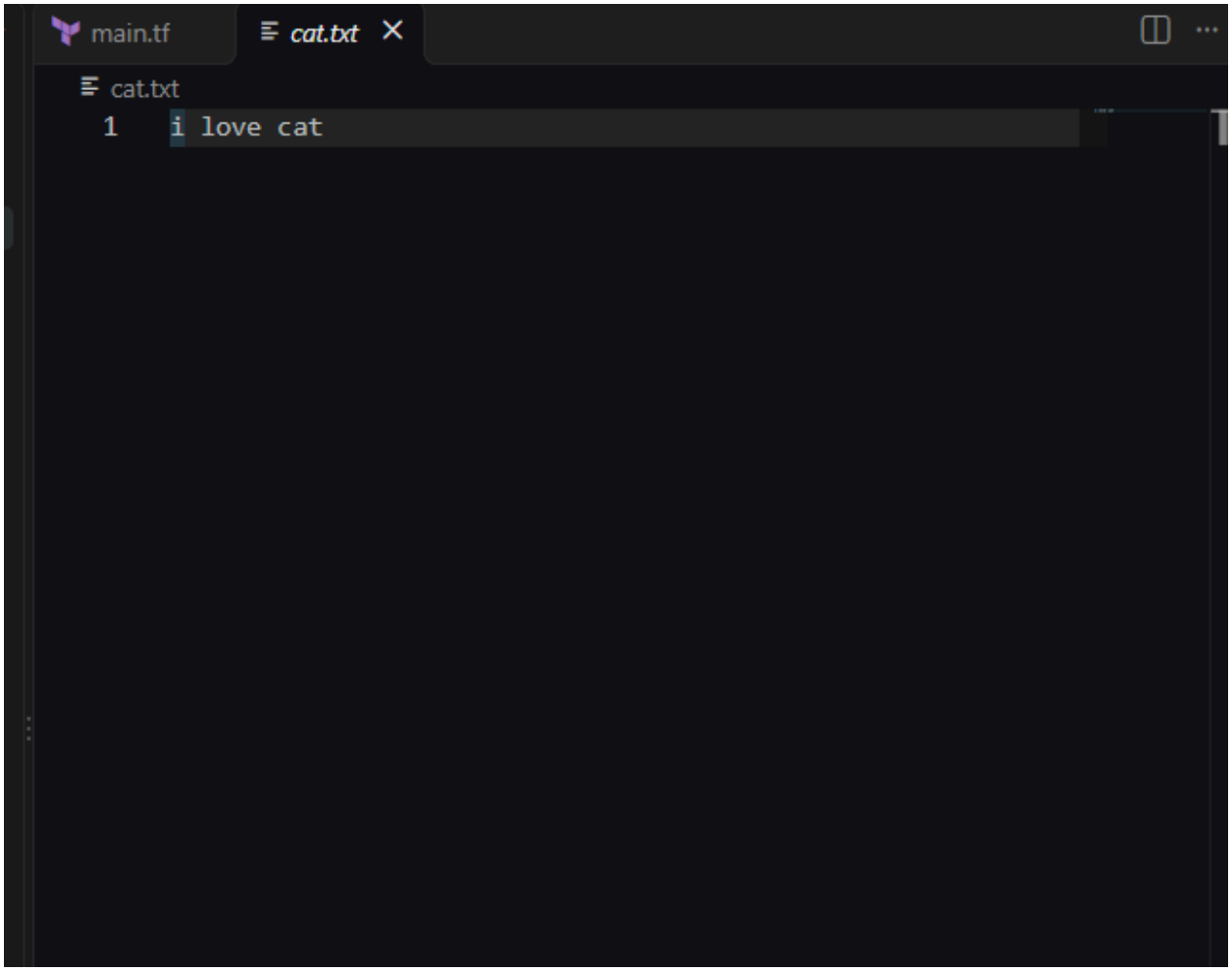
```
Lifecycle {
   Prevent_destroy =true
}
```

main.tf cat.txt

main.tf > resource "local_file" "my_pet" > content

```
1  resource "local_file" "my_pet" {
2  filename = "cat.txt"
3  content = "i love cat"
4  lifecycle {
5    prevent_destroy = true
6  }
7  }
8
9
```

For the first time it will executes
The output is i love cat



If we change any changes it will show an error

main.tf cat.txt

main.tf > resource "local_file" "my_pet" > content

```
1  resource "local_file" "my_pet" {
2  filename = "cat.txt"
3  content = "i love dogs"
4  lifecycle {
5    prevent_destroy = true
6  }
7  }
8
9
```

Problems Output Debug Console Terminal Ports powershell

```
Error: Instance cannot be destroyed

  on main.tf line 1:
   1: resource "local_file" "my_pet" {

Resource local_file.my_pet has lifecycle.prevent_destroy set, but the plan calls for this resource to be destroyed. To avoid this error and continue
with the plan, either disable lifecycle.prevent_destroy or reduce the scope of the plan using the -target option.

PS C:\terraform-practice>
```

3.ignore_changes

If we wat to ignore the changes when has been by third party tool or manually then we can use the ignore_changes.

The syntax is

```
Lifecycle {
   Ignore_changes[
     Content
]
}
```

main.tf cat.txt

main.tf > resource "local_file" "my_pet" > content

```
resource "local_file" "my_pet" {
filename = "cat.txt"
content = "i love dogsss........"
lifecycle {
  ignore_changes = [
    content
  ]
}
}
```

If we can apply the output will be no changes

```
PS C:\terraform-practice> terraform apply
local_file.my_pet: Refreshing state... [id=917192125050dceb1e7d211ea0165830e0946692]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\terraform-practice>
```

Task 2: Integrate Terraform in Jenkins Using the Terraform Plugin

1. Objective

To integrate Terraform with Jenkins using a Jenkins Pipeline and create the Terraform configuration file directly inside the Pipeline without creating `main.tf` manually.

2. Requirements

- Jenkins installed and running.
- Terraform installed on the Jenkins agent.

- Terraform plugin installed in Jenkins.
- Git is not required for this example.

3. Install the Terraform Plugin

1. Open the Jenkins Dashboard.
2. Click Manage Jenkins.
3. Select Plugins.
4. Search for Terraform under Available plugins.
5. Install the Terraform plugin.

4. Create a Jenkins Pipeline

1. Open the Jenkins Dashboard.
2. Click New Item.
3. Enter the name `terraform-pipeline`.
4. Select Pipeline and click OK.
5. Scroll down to the Pipeline section.
6. Select Pipeline script under Definition.
7. Paste the following Pipeline script.

5. Jenkins Pipeline Script

```
pipeline {
    agent any

    stages {
        stage('Check Terraform') {
            steps {
                sh 'terraform -version'
            }
        }

        stage('Create Terraform File') {
            steps {
                sh '''
                    cat > main.tf <<'EOF'
resource "local_file" "my_pet" {
  content  = "I love cats"
  filename = "pets.txt"
```

```
}
EOF
            '''
        }
    }

    stage('Initialize Terraform') {
        steps {
            sh 'terraform init'
        }
    }

    stage('Validate Terraform') {
        steps {
            sh 'terraform validate'
        }
    }

    stage('Plan Terraform') {
        steps {
            sh 'terraform plan'
        }
    }

    stage('Apply Terraform') {
        steps {
            sh 'terraform apply -auto-approve'
        }
    }
}

post {
    success {
        echo 'Terraform Pipeline completed successfully!'
    }

    failure {
        echo 'Terraform Pipeline failed. Check Console Output.'
    }
}
```

}

6. Explanation of the Pipeline

Stage	Purpose
Check Terraform	Checks the installed Terraform version.
Create Terraform File	Creates `main.tf` automatically using the shell command.
Initialize Terraform	Downloads the required provider and initializes the project.
Validate Terraform	Checks the Terraform configuration for errors.
Plan Terraform	Shows the changes Terraform intends to make.
Apply Terraform	Creates the resource defined in the configuration.

How does the file get created?

The following command creates the file directly from the Pipeline:

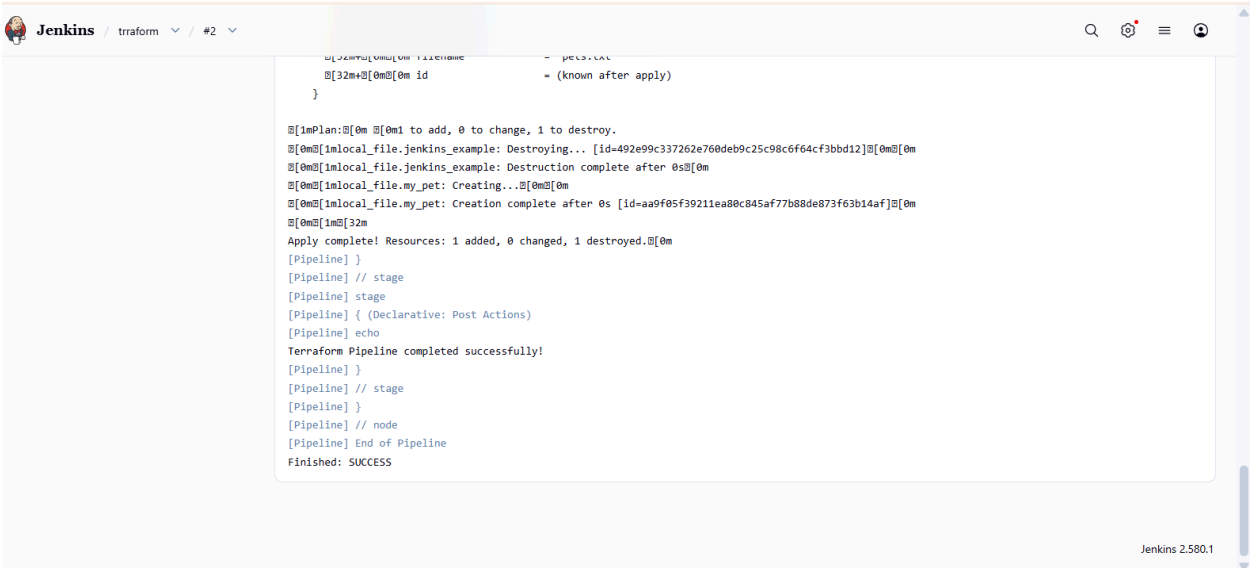
cat > main.tf <<'EOF'

The Terraform configuration written between `EOF` markers is saved into `main.tf`. You do not need to create the file manually.

The `local_file` resource creates another file named `pets.txt` containing `I love cats`.

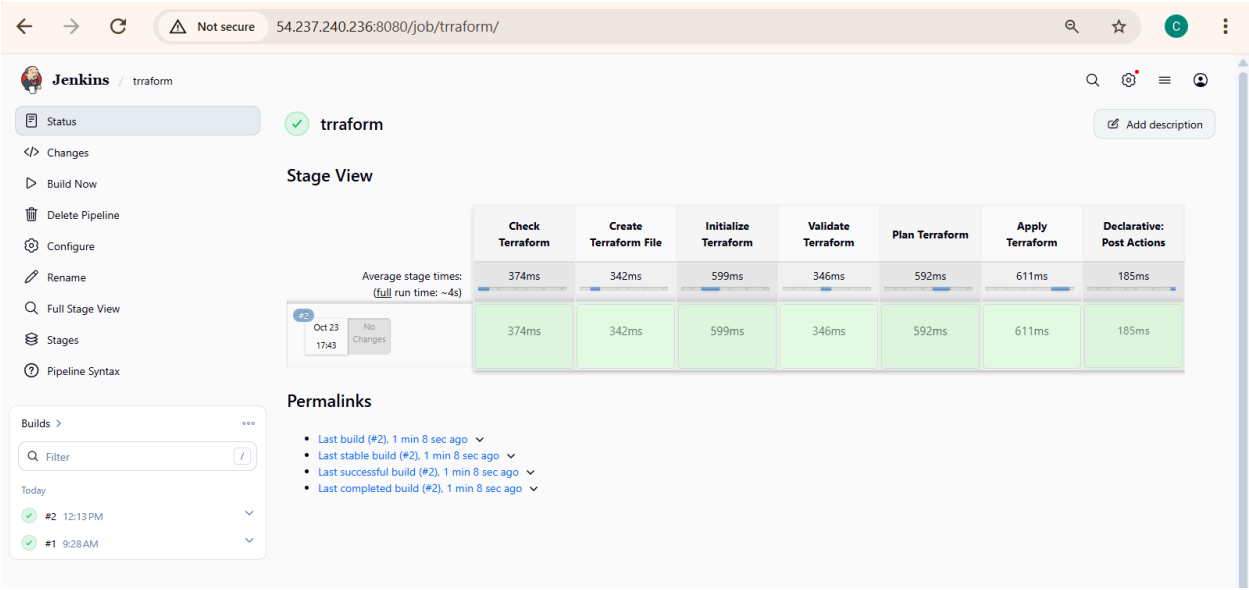
7. Execute the Pipeline

1. Click Save.
2. Open the `terraform-pipeline` job.
3. Click Build Now.
4. Open the build number.
5. Click Console Output.
6. Verify that all stages complete successfully.



Expected result:

- `main.tf` is created automatically inside the Jenkins workspace.
- Terraform initializes and validates the configuration.
- Terraform creates `pets.txt` in the workspace.
- Jenkins displays `Terraform Pipeline completed successfully!`



8. Result

Terraform was integrated with Jenkins, and the Terraform configuration file was created and executed entirely through a Jenkins Pipeline without manual file creation.

Note: This example uses Terraform CLI commands within Pipeline shell steps. The Terraform plugin is installed as required, but these shell commands do not themselves invoke plugin-specific Pipeline steps. The Jenkins agent must have Terraform installed and available in its PATH.